Build GPT

From Scratch, In Code, Spelled Out

A decoder-only Transformer built one piece at a time

A comprehensive, cell-by-cell study handbook for Andrej Karpathy's two-hour lecture “Let's build GPT: from scratch, in code, spelled out.” This expanded edition explains the **what**, the **why**, and the **how** behind every piece of the Transformer architecture — from character tokenization through scaled dot-product self-attention, multi-head attention, residual connections, LayerNorm, and the final decoder-only model that generates infinite (fake) Shakespeare.

Compiled by: ***Shreyas Pradeepkumar Khandale*** · *GitHub:* ***sherurox***

Companion to: `gpt_dev.ipynb` *and the nanoGPT repository*

Edition: Expanded - June 2026

Where this handbook sits relative to the makemore series

The five makemore lectures took a single character-level language model and refined it across a tight architectural ladder: a counting bigram (Part 1), a multilayer perceptron (Part 2), Kaiming initialization plus Batch Normalization (Part 3), manual backpropagation (Part 4), and a hierarchical WaveNet-style network (Part 5). By the end of that series we had crossed below 2.0 validation loss on the names dataset and built up a small PyTorch-style module library. The makemore lectures were a slow, careful tour of the fundamentals.

This lecture is different. It moves much faster, covers much more ground, and ends with a complete decoder-only Transformer — the architecture behind GPT-2, GPT-3, ChatGPT, and essentially every modern large language model. The dataset changes from names to tiny Shakespeare. The architecture changes from a flat or hierarchical MLP to a stack of Transformer blocks each containing multi-head self-attention, a position-wise feedforward network, residual connections, and LayerNorm. The model size changes from 76K parameters to about 10 million. This is the lecture that teaches you what is actually inside GPT.

What is actually new compared to the makemore series

- **The mathematical trick at the heart of attention.** Three escalating versions of weighted aggregation — explicit for-loop, lower-triangular matrix multiplication, and softmax with -infinity masking — that culminate in self-attention.
- **Keys, queries, and values.** Three learned linear projections that turn the constant uniform averaging of bigram models into data-dependent affinities. Tokens decide for themselves how much attention to pay to other tokens.
- **Scaled dot-product attention.** The $Q \cdot K^T / \sqrt{d_k}$ formula from *Attention Is All You Need*, with a careful explanation of why the scaling is needed (preventing softmax from saturating into a one-hot at initialization).
- **Multi-head attention.** Multiple attention heads running in parallel, concatenated channel-wise. The same model now learns multiple types of token-to-token communication simultaneously.
- **The Transformer block.** Multi-head self-attention plus a position-wise feedforward network, with residual connections wrapped around each. Communication followed by computation, repeated.
- **Residual connections.** The single innovation that makes deep stacks trainable. A gradient super-highway from the loss back to the input, with residual blocks gradually coming online as training progresses.
- **LayerNorm.** Normalization along the feature axis instead of the batch axis. Stateless, no train/eval distinction, and applied *before* the transformation in the pre-norm formulation we will adopt.
- **The encoder-decoder distinction.** Why our model is decoder-only (autoregressive, triangular masking) and what the original machine-translation paper used the encoder for (cross-attention to a separately processed input).
- **From pretraining to ChatGPT.** The full pipeline from pretraining a base model on the internet, through supervised fine-tuning, to Reinforcement Learning from Human Feedback — the three stages that turn a document completer into an assistant.

Preface: How to use this handbook

This handbook is a companion document for Karpathy's two-hour lecture “Let's build GPT: from scratch, in code, spelled out.” It walks through the lecture in the same order as the video, with each chapter mapping to a coherent group of cells in the `gpt_dev.ipynb` notebook. The format is identical to the five makemore handbooks (Parts 1-5), so the six documents can be read as a single coordinated tour of language modelling from counting bigrams to GPT-scale Transformers.

Reading mode 1: Linear study

Read the handbook front-to-back while keeping the notebook open in a separate window. Each chapter introduces a concept, presents the relevant cell verbatim from the notebook, and then walks through what the code is doing, why it is structured that way, and how it connects to the larger Transformer architecture. The lecture is dense — about thirty concepts in two hours — so giving each one its own short chapter prevents the compression that makes a single pass through the video so easy to get lost in.

Reading mode 2: Reference lookup

After a first pass, the handbook also works as a reference. The Table of Contents lists every concept by chapter, the Glossary defines every technical term in one line, and Chapter 28 collects the most common Transformer-specific pitfalls into diagnostic cards with symptom, diagnosis, and action rows. The cards are short and scannable; they exist to be flipped through when you are debugging code that should work but does not.

Conventions used in this handbook

The handbook uses a small set of visual conventions consistently throughout the document. They are identical to the conventions in the makemore handbooks.

- **Code blocks** appear in a fixed-width font on a warm beige background. Code is reproduced verbatim from the notebook so each block matches the cell that produced it.
- **Cell labels** precede every code block in small blue type (for example, *“Cell — The Bigram language model.”*).
- **Callout boxes** (yellow background, rust-coloured title) highlight Karpathy's spoken framing, important warnings, or moments of conceptual insight.
- **Info boxes** (light blue background) contain definitions and theoretical background. They are useful for first-time learners and skimmable for readers already familiar with the material.
- **Part dividers** separate the five major sections with a full-width banner.
- **Diagnostic cards** (Chapter 28) present each Transformer-specific pitfall in its own bounded card with three labeled rows: symptom, diagnosis, and action.

Prerequisites

This handbook assumes you are comfortable with the material covered in the makemore series — the multilayer perceptron, embeddings, cross-entropy loss, train/dev/test splits, BatchNorm, and (helpfully but not strictly required) manual gradient derivations from Part 4. Mathematical prerequisites are similar to Part 4: comfortable with the chain rule, matrix multiplications and their dimensions, softmax, and the basic syntax of PyTorch. Familiarity with the autoregressive language modelling framing — predict the next token given the previous tokens — is essential and is the bridge between the makemore series and this lecture.

How to extract maximum value from this material

The single most valuable thing you can take away from this lecture is the architecture of the Transformer itself — not just the mechanics of self-attention but the way the pieces fit together: tokens enter as embeddings, accumulate positional information, pass through a stack of communication-then-computation blocks each wrapped in residual connections, finally get normalized and projected to logits. Once you have internalized that pattern, you will recognize it in every modern paper. The specific hyperparameters change (model size, number of heads, vocabulary, training data) but the architecture is remarkably stable: the Transformer that ChatGPT runs on is structurally the same as the ten-million-parameter model we will build in this lecture.

Contents

PART I

From ChatGPT to a Bigram Baseline

Establishing what we are actually building, downloading tiny Shakespeare, and reproducing the makemore baseline.

1. The motivation: ChatGPT, GPT, and the Transformer

Chapter overview

Karpathy opens by grounding the lecture in something everyone has seen by now: ChatGPT. You give it a prompt, it generates text from left to right one token at a time, and the result is coherent enough that the world has spent the past few years adjusting to it. What is actually inside that system? The answer goes back to a 2017 paper from Google called *Attention Is All You Need*, which introduced the Transformer architecture. GPT — *Generatively Pre-trained Transformer* — is a specific application of that architecture to language modelling. Building the Transformer from scratch is the subject of this lecture.

Karpathy is careful to set the scope. We are not going to reproduce ChatGPT. ChatGPT is a production system trained on a substantial fraction of the internet, with extensive post-training (supervised fine-tuning and reinforcement learning from human feedback) on top of the base language model. What we will build is a much smaller character-level Transformer trained on tiny Shakespeare, with about 10 million parameters, that generates fake Shakespeare. The architecture is structurally identical to GPT-3 and ChatGPT — the difference is just scale (and the post-training, which we will not cover).

> **What "GPT" actually stands for**
>
> **G**eneratively **P**re-trained **T**ransformer. *Generative* because it produces output (as opposed to a classifier that just labels input). *Pre-trained* because it is first trained on a large unlabelled corpus before any task-specific fine-tuning. *Transformer* because the underlying neural network architecture is the Transformer from the 2017 paper. The combination produces a model that can be adapted to many tasks through prompting alone — the defining property of modern large language models.

The Attention Is All You Need paper

The 2017 paper introduced the Transformer in the specific context of machine translation. Karpathy notes that the paper itself reads "like a pretty random machine translation paper" — the authors did not anticipate the impact their architecture would have on every subfield of AI in the following five years. Image classification, audio, protein folding, code generation, robotics control — the Transformer has been copy-pasted (with minor modifications) into all of them. The version we will build is the decoder-only Transformer used for autoregressive language modelling, which is the version that powers GPT, Claude, Llama, and essentially every modern conversational AI.

> **Karpathy's framing**
>
> "What I'd like to focus on is just to train a Transformer-based language model, and in our case it's going to be a character-level language model. I still think that is very educational with respect to how these systems work. So I don't want to train on a chunk of Internet — we need a smaller dataset."

2. The tiny Shakespeare dataset

Chapter overview

The dataset for this lecture is “tiny Shakespeare” — a concatenation of all of Shakespeare's works into a single 1-megabyte text file, about 1.1 million characters. This is small enough to train on a single GPU in fifteen minutes and large enough to produce text that looks recognizably Shakespearean. It is Karpathy's favourite toy dataset and it appears across many of his tutorials.

Cell - Download and inspect the dataset

```
!wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

with open('input.txt', 'r', encoding='utf-8') as f:
    text = f.read()

print('length of dataset in characters:', len(text))   # 1115394
print(text[:1000])
```

The first 1000 characters look like what you would expect: dialogue tags, speech, blank lines, the structural elements of Shakespeare's plays. The whole file follows that pattern, with all of his plays concatenated together. This is exactly the kind of text we want to model: rich enough to have interesting structure (vocabulary, line breaks, punctuation, character names) but small enough to train on quickly.

> **Why character-level instead of word- or subword-level**
> Modern language models use subword tokenization — SentencePiece (Google), BPE (OpenAI's tiktoken), and similar. Subword tokens encode common letter sequences as single tokens, producing much shorter sequences per character of input. This lecture uses character-level tokenization because it is simpler to explain and lets us see the model's predictions one character at a time. The architecture would be identical with subword tokens — just larger vocabulary and shorter sequences.

3. Character-level tokenization

Chapter overview

Before any neural network can process text, the text has to be converted to integers. This is called tokenization. For our character-level model, the tokenization scheme is the simplest possible: every distinct character in the dataset gets its own integer, and we encode strings as lists of those integers. This chapter introduces the encoder and decoder functions.

Cell - Build the vocabulary and encoder/decoder

```
chars = sorted(list(set(text)))
vocab_size = len(chars)
print(''.join(chars))   # all unique characters in the dataset
print(vocab_size)       # 65

# create a mapping from characters to integers
```

```
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for i, ch in enumerate(chars)}
encode = lambda s: [stoi[c] for c in s]
decode = lambda l: ''.join([itos[i] for i in l])

print(encode('hii there'))                # [46, 47, 47, 1, 58, 46, 43, 56, 43]
print(decode(encode('hii there')))        # 'hii there'
```

The vocabulary has 65 characters: space, newline, punctuation, the digits, and both uppercase and lowercase letters. The character ordering is arbitrary — we just sort the unique characters and number them from 0 onwards. The newline character gets index 0 (it sorts first), which will matter later because we use index 0 as the special “start of generation” token when sampling.

Tradeoffs of different tokenization schemes

Karpathy briefly mentions other tokenization approaches:

- **Character-level (ours).** Vocabulary of 65. Very long sequences (one token per character). Simplest possible. Used in tutorials and small-scale experiments.
- **Word-level.** Vocabulary of tens of thousands. Short sequences. But struggles with out-of-vocabulary words and morphology.
- **Subword-level (SentencePiece, BPE, WordPiece).** Vocabulary of about 50,000 (in OpenAI's case). Sequences of moderate length. Best of both worlds and what real production systems use.

> **A small vocabulary means long sequences, and vice versa**
> There is a fundamental tradeoff in tokenization between vocabulary size and sequence length. A character-level model with 65 tokens has very long sequences but a very small input/output layer. A word-level model with 50,000 tokens has shorter sequences but a much larger embedding and output layer. Subword tokenization, used by every modern LLM, is the practical middle ground.

4. Encoding the dataset and the train/val split

Chapter overview

Once we have the tokenizer, we encode the entire 1.1-million-character dataset into a single PyTorch tensor of integers and then split it into training and validation sets. This chapter is a brief one, but it sets up the structures that every subsequent training and evaluation step will pull from.

Cell - Encode and split

```
import torch
data = torch.tensor(encode(text), dtype=torch.long)
print(data.shape, data.dtype)   # torch.Size([1115394]) torch.int64

# 90/10 train/val split
n = int(0.9 * len(data))
train_data = data[:n]
val_data = data[n:]
```

The dataset becomes a 1D tensor of 1.1 million integers. The first 90% (about 1 million characters) is the training set; the last 10% is the validation set. The split is just by position — we do not shuffle and we do not stratify in any way. For a small stationary dataset like Shakespeare this is fine, though for production systems the splitting strategy is much more carefully designed.

> **Why no test set in this lecture**
> Karpathy uses train/val only, not train/val/test. The makemore series used train/dev/test. In a pedagogical setting like this one the distinction does not matter much — we are not doing serious hyperparameter selection. In a real research project, the test set is held out completely until the very end to give an unbiased estimate of generalization. For this lecture, the validation set serves both roles.

5. Batches, blocks, and the data loader

Chapter overview

We do not train on the entire 1-million-character training set at once. We sample random minibatches of fixed-length context windows. Each window is called a *block*, and we process several blocks in parallel as a *batch*. This chapter introduces the block size and batch size hyperparameters and the data loader function that produces minibatches.

Cell - Get a random batch

```
torch.manual_seed(1337)
batch_size = 4    # how many independent sequences will we process in parallel
block_size = 8    # maximum context length for predictions

def get_batch(split):
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i + block_size]   for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    return x, y

xb, yb = get_batch('train')
print(xb.shape)   # torch.Size([4, 8])
print(yb.shape)   # torch.Size([4, 8])
```

Reading the data loader

- **block_size = 8** — the maximum context length. The model sees at most 8 previous characters when predicting the 9th. In the full model we will increase this to 256.
- **batch_size = 4** — how many independent windows we process per training step. In the full model we will increase this to 64.
- `ix` — `batch_size` random starting positions sampled uniformly from the training data.

- `x` — the input contexts. Each row is 8 consecutive characters starting at one of the random positions. Shape (B, T) = (4, 8).
- `y` — the targets. Each row is the same 8 positions shifted by one — so `y[b, t]` is the character that should follow `x[b, t]`.

> **Why every position in a block is a training example**
> An (x, y) pair of shape (T,) is not just one prediction problem — it is T prediction problems. Given `x[0]`, predict `y[0]`. Given `x[:2]`, predict `y[1]`. Given `x[:3]`, predict `y[2]`. And so on, all the way up to: given `x[:T]`, predict `y[T-1]`. Each block provides T separate training examples that share most of their computation. This is why training is so efficient — one forward pass produces T loss terms.

> **B, T, C notation**
> Throughout the rest of the lecture, we use three letter conventions for tensor dimensions: B for *batch*, T for *time* (the sequence/position dimension), and C for *channels* (the feature/embedding dimension). Every intermediate tensor in the Transformer has shape (B, T, C). Keeping this convention in mind makes shape errors much easier to diagnose.

6. The Bigram baseline: train and generate

Chapter overview

Before building the Transformer, Karpathy starts with a baseline that should already be familiar from the makemore series: a bigram language model. Every token directly looks up its logits for the next token in a table, with no context. This is the absolute simplest language model that still has parameters, and it gives us a working forward pass, training loop, and generation function that we will gradually upgrade into a full Transformer.

Cell - The Bigram language model

```
import torch.nn as nn
from torch.nn import functional as F

class BigramLanguageModel(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        # each token directly reads off the logits for the next token
        self.token_embedding_table = nn.Embedding(vocab_size, vocab_size)

    def forward(self, idx, targets=None):
        logits = self.token_embedding_table(idx)   # (B, T, C)
        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
            targets = targets.view(B*T)
```

```
            loss = F.cross_entropy(logits, targets)
        return logits, loss

    def generate(self, idx, max_new_tokens):
        for _ in range(max_new_tokens):
            logits, loss = self(idx)
            logits = logits[:, -1, :]              # focus on the last time step
            probs = F.softmax(logits, dim=-1)      # (B, C)
            idx_next = torch.multinomial(probs, num_samples=1)  # (B, 1)
            idx = torch.cat((idx, idx_next), dim=1)
        return idx
```

How the bigram model works

The embedding table has shape `(vocab_size, vocab_size)` = (65, 65). When we pass in a (B, T) tensor of integer indices, PyTorch plucks out the corresponding rows and arranges them into a (B, T, C) tensor where C = vocab_size = 65. We interpret those values as the *logits* — raw, unnormalized scores — for the next character. The bigram model uses only the identity of the current token to predict the next one; no context, no attention, nothing. It is just a learned lookup table.

> **Why the .view() before cross_entropy**
> PyTorch's `F.cross_entropy` expects logits in a specific shape. For multi-dimensional input, it wants channels as the *second* dimension (B, C, T) rather than the last dimension (B, T, C). To avoid that convention mismatch, we reshape our logits to 2D — `(B*T, C)` — and reshape targets to 1D — `(B*T,)`. Cross-entropy then treats each of the B*T positions as an independent classification problem and averages over all of them.

The expected initial loss

With 65 classes and random initialization, we expect cross-entropy loss `−log(1/65) ≈ 4.17` at the start of training. Karpathy observes 4.87, slightly worse than uniform because the initialization is not exactly uniform. After 10,000 training iterations with AdamW and learning rate 1e-3, the loss drops to about 2.5 — the model has learned which characters typically follow which other characters, but the generations are gibberish because there is no context.

Why AdamW instead of SGD

The makemore series used plain stochastic gradient descent — the simplest possible optimizer, which just steps every parameter by `−lr * grad`. For this lecture Karpathy switches to AdamW, which is “a much more advanced and popular optimizer” that has become the default for Transformer training. AdamW maintains per-parameter running estimates of the gradient mean and variance and uses those to scale each parameter's update individually. The effect is that parameters with consistently small gradients get larger effective learning rates while parameters with noisy gradients get smaller ones, which dramatically accelerates convergence on the loss surfaces that Transformers tend to produce.

Karpathy's rule of thumb for learning rates

“A typical good setting for the learning rate is roughly 3e-4, but for very very small networks like is the case here, you can get away with much much higher learning rates — 1e-3 or even higher probably.” The 3e-4 figure is so well-known in deep learning practice that it has become a meme (the “Karpathy constant”). Smaller networks tolerate higher learning rates because the loss surface is simpler; bigger networks need smaller rates because the curvature is more delicate.

Cell - Train and generate

```
optimizer = torch.optim.AdamW(m.parameters(), lr=1e-3)

batch_size = 32
for steps in range(10000):
    xb, yb = get_batch('train')
    logits, loss = m(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

print(loss.item())   # about 2.48

# generate 500 characters starting from a single newline
context = torch.zeros((1, 1), dtype=torch.long)
print(decode(m.generate(context, max_new_tokens=500)[0].tolist()))
```

Why the generate function is written generally even for a bigram

“This function is written to be general but it's kind of ridiculous right now, because we're feeding in all this context and we're concatenating it all and we're always feeding it all into the model. But that's kind of ridiculous because this is just a bigram model. The only reason I'm writing it in this way is because right now this is a bigram model, but I'd like to keep this function fixed and have it work later when our characters actually look further in the history.” The same generate function will serve the full Transformer at the end of the lecture without any changes.

PART II

The Mathematical Trick at the Heart of Attention

Three escalating versions of weighted aggregation that build up from a triple for-loop to a single matrix multiplication.

7. The motivation: tokens need to talk

Chapter overview

The bigram model is limited because each token sees only itself when predicting the next token. The fifth token in a sequence cannot incorporate information from the first four tokens. To do better, the tokens need to communicate — the fifth token needs to "see" the previous tokens and aggregate information from them. But there is a crucial constraint: information must flow only from past to future, never the other way around, because at inference time we will not have access to future tokens.

The constraint, stated precisely

- Token at position 0 sees only itself.
- Token at position 1 sees positions 0 and 1.
- Token at position 2 sees positions 0, 1, and 2.
- ...
- Token at position T-1 sees all T positions.

This pattern — each position seeing itself and all earlier positions but no later ones — is called *causal masking* or *autoregressive masking*. It is what makes the Transformer a language model rather than a bidirectional encoder.

> **Why the past-to-future constraint matters**
> If we let tokens see future positions during training, the network would learn to "cheat" by copying information from the very token it is supposed to predict. The training loss would go to zero (perfectly predict every token by looking at it directly), but at inference time we do not have access to future tokens, so the model would produce nonsense. The causal mask enforces a self-consistent training objective.

The simplest possible communication: averaging

Karpathy proposes the simplest possible form of token-to-token communication: averaging. The fifth token replaces its representation with the average of representations at positions 0 through 5 (itself and all earlier positions). This is what is sometimes called "bag of words" aggregation — it preserves nothing about the order or individual identity of the tokens being averaged, but it gets us a working baseline that we can refine.

“Extremely lossy but okay for now”
“Just doing a sum or like an average is an extremely weak form of interaction. Like this communication is extremely lossy — we've lost a ton of information about the spatial arrangements of all those tokens. But that's okay for now — we'll see how we can bring that information back later.” The first version (averaging) gets us through the mechanics of weighted aggregation. The fourth version (full attention) fixes the loss of information.

8. Version 1: explicit averaging with nested for-loops

Chapter overview

Karpathy implements the past-averaging operation in the simplest possible way first: with explicit nested for-loops. This is slow and inelegant but unambiguous, and it gives us a ground-truth result to compare the more efficient versions against in Chapters 9 and 10.

Cell - Toy example setup

```
torch.manual_seed(1337)
B, T, C = 4, 8, 2   # batch, time, channels
x = torch.randn(B, T, C)
x.shape   # torch.Size([4, 8, 2])
```

A small toy example: 4 batches, 8 positions per batch, 2 channels (features) per position. The values are random Gaussians. Our goal is to compute a new tensor `xbow` where `xbow[b, t]` is the mean of `x[b, 0:t+1]` — the average of all positions from 0 through t inclusive.

Cell - Version 1: explicit averaging with for-loops

```
# Goal: xbow[b, t] = mean of x[b, 0:t+1] across the time dimension
xbow = torch.zeros((B, T, C))
for b in range(B):
    for t in range(T):
        xprev = x[b, :t+1]              # shape (t+1, C)
        xbow[b, t] = torch.mean(xprev, 0)
```

The outer loop iterates over batches; the inner loop iterates over positions. At each (batch, position) pair we extract the slice of `x` up to and including that position and take its mean. The result is the past-averaged version of `x`.

What the result looks like
For the zeroth position, `xbow[b, 0]` equals `x[b, 0]` (the mean of one element is itself). For the first position, `xbow[b, 1]` equals `(x[b, 0] + x[b, 1]) / 2`. For the last position, `xbow[b, T-1]` equals the mean of all T positions in row b. Each new position blends in one more element of the prefix, so later positions have more “contributing” tokens in their average.

Why this is too slow

For B = 4 and T = 8 the loops run 32 times. For realistic B = 64 and T = 256 they would run over 16,000 times, each iteration doing a sum and a division. The whole computation is mathematically a sequence of weighted sums — exactly the kind of thing matrix multiplication is designed for. The next chapter shows how to do all 32 averages in a single matrix multiplication.

9. Version 2: matrix multiplication as weighted aggregation

Chapter overview

This chapter introduces the mathematical trick that unlocks self-attention: a lower-triangular matrix multiplied with our input tensor produces exactly the weighted aggregation we want, all in one operation. The chapter walks through the intuition with a 3×3 toy example before scaling up.

The toy example

Cell - Why matrix multiplication does what we want

```
torch.manual_seed(42)
a = torch.tril(torch.ones(3, 3))             # lower-triangular ones
a = a / torch.sum(a, 1, keepdim=True)        # normalize each row to sum to 1
b = torch.randint(0, 10, (3, 2)).float()
c = a @ b

print(a)
# tensor([[1.0000, 0.0000, 0.0000],
#         [0.5000, 0.5000, 0.0000],
#         [0.3333, 0.3333, 0.3333]])

print(b)
# tensor([[2., 7.],
#         [6., 4.],
#         [6., 5.]])

print(c)
# tensor([[2.0000, 7.0000],
#         [4.0000, 5.5000],
#         [4.6667, 5.3333]])
```

Reading the result

The output row `c[i]` is a weighted sum of the rows of `b`, with the weights given by row `a[i]`. Because row `a[0]` is `[1, 0, 0]`, `c[0]` is just `b[0]`. Because row `a[1]` is `[0.5, 0.5, 0]`, `c[1]` is the average of `b[0]` and `b[1]`. Because row `a[2]` is `[1/3, 1/3, 1/3]`, `c[2]` is the average of all three rows of `b`. **The matrix multiplication computed all three prefix averages in a single operation.**

The pattern, stated precisely

If a is lower-triangular with rows that sum to 1, then a `@` b produces, in each row of the output, the weighted average of all rows of b up to and including the current position. The weights come from the corresponding row of a. Different patterns in a produce different aggregations — uniform averaging, exponential decay, attention to specific positions, anything.

Applying it to our (B, T, C) tensor

Cell - Version 2: batched matrix multiplication

```
wei = torch.tril(torch.ones(T, T))
wei = wei / wei.sum(1, keepdim=True)
xbow2 = wei @ x                          # (T, T) @ (B, T, C) -> (B, T, C)

torch.allclose(xbow, xbow2)              # True
```

PyTorch broadcasts the (T, T) matrix across the batch dimension automatically, applying the matrix multiplication independently to each of the B batch elements in parallel. The result `xbow2` is identical to the for-loop result `xbow` from Chapter 8, but the computation is one tensor operation instead of B×T iterations. This is a substantial speedup, and on a GPU the gap is even more dramatic.

The trick, in one sentence

You can do prefix averaging by multiplying with a normalized lower-triangular matrix. Different patterns in that matrix produce different aggregations. The next chapter shows how to generate the weights from a different starting point that will let us make them data-dependent.

10. Version 3: softmax with -infinity masking

Chapter overview

Version 2 hard-codes uniform averaging via the lower-triangular ones matrix. Version 3 rewrites that exact same uniform-averaging behaviour in a different form — using softmax with `-inf` masking — that will be much more flexible. The reason for the rewrite is that softmax can take *any* input tensor and produce a normalized, lower-triangular weighting from it. Once that input tensor can be computed from data, the weights become data-dependent — which is exactly what self-attention will do in Part III.

Cell - Version 3: softmax with mask

```
tril = torch.tril(torch.ones(T, T))
wei = torch.zeros((T, T))
wei = wei.masked_fill(tril == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)
xbow3 = wei @ x

torch.allclose(xbow, xbow3)   # True
```

Walking through the three lines

- `wei = torch.zeros((T, T))` — start with all zeros. Currently uniform, but soon this will hold learned interaction strengths called *affinities*.
- `wei.masked_fill(tril == 0, -inf)` — set every position in the upper triangle to `–inf`. These are the "future" positions that should not communicate to the current position.
- `F.softmax(wei, dim=-1)` — exponentiate and normalize each row. Since `exp(–inf) = 0`, the future positions contribute zero weight. Since the lower-triangular entries are all zero, `exp(0) = 1`, and the row normalization gives them uniform weight.

> **Why this rewrite is the key step**
> Version 2 produced uniform weights by hard-coding them. Version 3 produces the exact same uniform weights by passing zeros through softmax. The crucial difference is that softmax can take *any* values in those starting positions — not just zeros — and still produce a normalized distribution where the upper triangle is masked. When we replace those zeros with values computed from data, we get data-dependent weights. The `–inf`-masking-then-softmax pattern is the universal way to express "weighted average over the past" in modern Transformer code.

Why this version is so important

Think of the starting `wei` matrix as expressing how strongly each token wants to listen to each other token. In version 3, those interaction strengths are zero everywhere — we are saying every token cares about every past token equally. In self-attention, those strengths will be computed from data. Some tokens will find other tokens very interesting (high affinity), and the softmax will give them most of the aggregation weight. Other tokens will be ignored (low affinity). The structure of softmax-with-mask-and-aggregation stays exactly the same; only the source of the pre-softmax values changes.

11. Why this matters: from constant to data-dependent weights

Chapter overview

This brief chapter consolidates what we have established in Chapters 7–10 and previews what self-attention will add on top. The mathematical trick of the previous three chapters is genuinely the entire skeleton of attention — what attention adds is just a specific way of computing the pre-softmax affinity values from the input tokens.

The three ingredients we have

- **A way to aggregate from past tokens.** Matrix multiplication with a lower-triangular weighting matrix gives every position a weighted sum of past positions.
- **A way to enforce the causal constraint.** Setting the upper triangle to `–inf` before softmax zeros out the contribution of future positions exactly.

- **A way to normalize the weights.** Softmax converts arbitrary real-valued affinities into a probability distribution over past positions, so the weights always sum to 1.

The one ingredient we are missing

The pre-softmax values are still hardcoded to zero. What we want is for the model to *learn*, from the data, how interested each token is in each other token. The fifth token reading the third token might find it extremely relevant (because the third token is a noun and the fifth token is an adjective looking for a noun to modify) and almost ignore the fourth token. The fifth token reading itself might pay very little attention to itself if its own information is already represented elsewhere. Whatever the optimal pattern is, we want it to emerge from training rather than be hardcoded.

> **The preview, from Karpathy**
> “These zeros are currently just set by us to be zero, but a quick preview is that these affinities between the tokens are not going to be just constant at zero. They're going to be data-dependent. These tokens are going to start looking at each other and some tokens will find other tokens more or less interesting, and depending on what their values are they're going to find each other interesting to different amounts. I'm going to call those affinities.”

Part III introduces the keys-queries-values mechanism that produces these data-dependent affinities. The aggregation machinery stays exactly as we have built it; what changes is where the pre-softmax values come from.

PART III

Self-Attention from Scratch

Token and positional embeddings, keys, queries, values, and the famous scaled dot-product attention formula.

12. Token and positional embeddings

Chapter overview

Before introducing self-attention proper, Karpathy refactors the bigram model. The embedding table no longer maps directly to vocab-sized logits — it maps to a lower-dimensional "n_embd" space (32 in this version, 384 in the final). A separate *language modelling head* linear layer projects from that intermediate space back up to vocab_size. The refactor introduces an intermediate representation that we will soon stuff full of attention and feedforward operations.

Karpathy also adds a second embedding table: positional embeddings. Each position in the block (0 through block_size–1) gets its own learnable n_embd-dimensional vector. The token embedding tells the model *what* character is at this position; the positional embedding tells it *where* in the sequence it is.

Cell - Token + positional embeddings

```
class BigramLanguageModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_embedding_table    = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        tok_emb = self.token_embedding_table(idx)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device))
        x = tok_emb + pos_emb                  # (B, T, n_embd)
        logits = self.lm_head(x)               # (B, T, vocab_size)
        ...
```

Reading the shapes

- `tok_emb` has shape (B, T, n_embd). For each example in the batch and each time step, we look up an n_embd-dimensional vector representing the character at that position.
- `pos_emb` has shape (T, n_embd). We embed the integer positions 0, 1, ..., T-1 into n_embd-dimensional vectors. Same vectors for every example in the batch, so there is no B dimension.
- `x = tok_emb + pos_emb` — PyTorch broadcasts the (T, n_embd) positional embeddings across the batch dimension and adds them element-wise to the token embeddings. The result has shape (B, T, n_embd) and encodes both the identity and the position of every token.
- `lm_head(x)` projects from (B, T, n_embd) to (B, T, vocab_size) to produce logits.

Why positional embeddings are necessary

Attention by itself has no notion of position — it treats the input as a set of vectors, not a sequence. If two tokens are identical, attention cannot tell which one came first. Adding a learned position-specific vector to each token's embedding breaks that symmetry. There are several ways to do this in the literature: learned absolute positions (what we use), sinusoidal positions (in the original paper), RoPE (used by many modern LLMs), and ALiBi (used by some others). Learned absolute positions are the simplest and work fine for short contexts.

A required modification to generate()

Adding positional embeddings introduces a subtle requirement on the generate function. The position embedding table has only `block_size` entries — one for each valid position. During generation, the running context grows by one token every iteration. Once it exceeds `block_size`, asking the position embedding table for the embedding at position `block_size` or beyond will throw an index-out-of-range error. We have to crop the context to its last `block_size` tokens before each forward pass.

Cell - Generate with context cropping

```
def generate(self, idx, max_new_tokens):
    for _ in range(max_new_tokens):
        # crop idx to the last block_size tokens
        idx_cond = idx[:, -block_size:]
        logits, loss = self(idx_cond)
        logits = logits[:, -1, :]
        probs = F.softmax(logits, dim=-1)
        idx_next = torch.multinomial(probs, num_samples=1)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx
```

The single line `idx_cond = idx[:, -block_size:]` takes the last `block_size` tokens of the current context. If the context is shorter than `block_size`, slicing returns the whole thing — no harm done. If it is longer, the older tokens are silently dropped. The model becomes structurally limited to a fixed maximum context, and any information beyond that window is forgotten.

Why this matters at every Transformer scale

This same fixed-context-window limitation applies to every Transformer, including GPT-4 with its 128K context. The window is much bigger but it is still a window. Once a conversation exceeds the model's context size, older tokens are dropped (in most implementations) or summarized into a shorter form (in others). The structural constraint that we hit here on character 9 of generation is the same constraint that very large LLMs hit on token 128001.

13. The Head module: keys, queries, and values

Chapter overview

Self-attention introduces three learned linear projections of the input. Every token produces three vectors: a *key*, a *query*, and a *value*. Conceptually, the query is "what I am looking for," the key is "what I have to offer," and the value is "what I will communicate if you find me interesting." This chapter introduces the three projections and the intuition for their roles.

Cell - Single head of self-attention (toy)

```
B, T, C = 4, 8, 32   # batch, time, channels
x = torch.randn(B, T, C)

# a single Head of self-attention
head_size = 16
key   = nn.Linear(C, head_size, bias=False)
query = nn.Linear(C, head_size, bias=False)
value = nn.Linear(C, head_size, bias=False)

k = key(x)     # (B, T, 16)
q = query(x)   # (B, T, 16)
v = value(x)   # (B, T, 16)
```

Three linear projections, three different roles

- **Key** — what this token contains. If another token's query matches this key, the other token will pay attention here.
- **Query** — what this token is looking for. The query is dotted with every other token's key to determine attention weights.
- **Value** — what gets communicated when attention is paid. The actual content that gets aggregated, separate from the key (which is just for matching purposes).

> **The private-information metaphor**
> "You can think of x as kind of like private information to this token. So x is kind of private to this token. I'm a fifth token at some position and I have some identity and my information is kept in vector x. And now for the purposes of the single head: here's what I'm interested in (query), here's what I have (key), and if you find me interesting, here's what I will communicate to you (value)." This three-part framing is fundamental to understanding why attention works.

Notice that all three projections have `bias=False`. This is the standard convention in attention layers. The projections are essentially rotating the input vectors into three different subspaces, and a bias would just shift the result, which is not what we want here.

14. Computing affinities via Q @ K-transpose

Chapter overview

Once we have keys and queries for every position, computing the affinity between every pair of positions is a single matrix multiplication. The query of position *i* is dotted with the key of position *j* to get the affinity score for "how much should position *i* attend to position *j*." Done for all pairs in parallel, this is the matrix

product `q @ k.transpose(-2, -1)`.

Cell - Computing all pairwise affinities

```
wei = q @ k.transpose(-2, -1)   # (B, T, 16) @ (B, 16, T) -> (B, T, T)
```

Reading the shapes

q has shape (B, T, head_size). k has the same shape. We transpose the last two dimensions of k to get (B, head_size, T), then matmul produces (B, T, T). PyTorch handles the batch dimension automatically — for each of the B examples in the batch, we get a T×T matrix of pairwise affinities. Entry `wei[b, i, j]` is the dot product of position *i*'s query with position *j*'s key in batch element b.

> **Why dot product**
> The dot product of two vectors is a natural similarity score: it is large and positive when the vectors point in the same direction, large and negative when they point in opposite directions, and near zero when they are orthogonal. By computing `query @ key`, we ask: how aligned is what this token is looking for with what that token has to offer? Aligned queries and keys produce large positive values that, after softmax, become large attention weights. Misaligned queries and keys produce near-zero attention weights.

From affinities to attention weights

Right now `wei` is just a tensor of dot-product values, ranging anywhere from large negative to large positive. To turn these into attention weights, we need to apply two more operations — masking (Chapter 15) and softmax. Both are imported from the version 3 mathematical trick of Chapter 10.

15. Triangular masking for autoregressive generation

Chapter overview

The masking step enforces our causal constraint: position *i* can only attend to positions *0..i*, never to positions *i+1..T-1*. We achieve this by setting every entry above the diagonal of `wei` to `–inf` before applying softmax. After softmax, those positions get exactly zero weight.

Cell - Apply the causal mask

```
tril = torch.tril(torch.ones(T, T))
wei = wei.masked_fill(tril == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)   # (B, T, T)
```

After softmax, each row of `wei` is a probability distribution over the past positions. The first row puts all its weight on position 0, because that is the only allowed position. The second row distributes weight between positions 0 and 1. The last row distributes weight across all T positions. **The weights are now data-dependent** — they depend on the input via q and k, which are computed from the input embeddings.

The encoder/decoder distinction starts here

If you delete the masking line, every position can attend to every other position — future included. This is called an *encoder* block (BERT-style). With the masking line, future positions are forbidden — this is a *decoder* block (GPT-style). The choice depends entirely on what you are doing: classification over a known input uses an encoder; autoregressive generation uses a decoder. Our model is a decoder, hence the mask.

16. Aggregating with the value vector

Chapter overview

The final step of a single attention head is to use the attention weights to aggregate the *value* vectors (not the raw input x). The values are the third linear projection of the input, and they represent "what this token will communicate." Aggregating values rather than raw inputs gives the model an extra degree of freedom — the same token can offer different things to different queries.

Cell - Aggregate values

```
v = value(x)           # (B, T, head_size)
out = wei @ v          # (B, T, T) @ (B, T, head_size) -> (B, T, head_size)
```

The output has the same shape as q, k, and v — (B, T, head_size). Each position now contains a weighted combination of value vectors from itself and all earlier positions, weighted by the data-dependent attention weights. The output is the token's new representation after the head has communicated with the past.

Why aggregate values instead of x directly

If we aggregated raw x instead of v, we would be saying that every token communicates its full information whenever queried. The value projection lets the token decide what part of itself to expose. A token might have one piece of information that is relevant for queries about syntax and another for queries about meaning; the value projection learns to project out the relevant subspace. Concretely, the value matrix is just another learnable linear layer, and training will set it to whatever projection best minimizes the loss.

Putting it all together: a single attention head

Cell - The Head module (final form)

```
class Head(nn.Module):
    """ one head of self-attention """
    def __init__(self, head_size):
        super().__init__()
        self.key   = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)
```

```
    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x)
        q = self.query(x)
        # compute attention scores ('affinities')
        wei = q @ k.transpose(-2, -1) * C ** -0.5      # (B, T, T)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
        wei = F.softmax(wei, dim=-1)
        wei = self.dropout(wei)
        # perform the weighted aggregation of the values
        v = self.value(x)
        out = wei @ v
        return out
```

Three linear projections, a matmul, a mask, a softmax, and a final matmul. That is the entire self-attention mechanism. The `register_buffer` stores the triangular mask as a non-parameter tensor that moves with the model between CPU and GPU but does not get updated by the optimizer. The `* C ** -0.5` is the scaling factor we explain in the next chapter.

17. Scaled dot-product attention

Chapter overview

There is one more detail that makes attention work robustly: dividing the affinities by the square root of the head size before softmax. This is the “scaled” part of “scaled dot-product attention” in the original paper. The reason is statistical: without scaling, the variance of the dot products grows linearly with the head size, which causes softmax to saturate into a one-hot distribution at initialization. This chapter explains the problem and the fix.

The variance problem

If q and k have unit-variance entries (e.g., from standard initialization), the dot product `q[i] @ k[j]` is a sum of *head_size* products of unit-variance Gaussians. The result has variance roughly equal to *head_size*. For head_size = 16, the affinities have standard deviation 4. For head_size = 64, standard deviation 8. The larger the head, the more extreme the pre-softmax values.

Cell - Demonstrating the variance issue

```
k = torch.randn(B, T, head_size)
q = torch.randn(B, T, head_size)

wei_unscaled = q @ k.transpose(-2, -1)
print(wei_unscaled.var())   # about head_size (e.g. 16)

wei_scaled = q @ k.transpose(-2, -1) * head_size ** -0.5
print(wei_scaled.var())     # about 1.0
```

Why softmax saturation is bad

Cell - Showing softmax saturation

```
# diffuse - small values - close to uniform
torch.softmax(torch.tensor([0.1, -0.2, 0.3, -0.2, 0.5]), dim=-1)
# tensor([0.1925, 0.1426, 0.2351, 0.1426, 0.2872])

# peaky - same values scaled by 8 - approaches one-hot
torch.softmax(torch.tensor([0.1, -0.2, 0.3, -0.2, 0.5]) * 8, dim=-1)
# tensor([0.0212, 0.0019, 0.1048, 0.0019, 0.8702])
```

When the pre-softmax values are small, softmax produces a diffuse distribution — every position gets meaningful weight. When the values are large, softmax converges toward a one-hot vector — almost all the weight goes to the single largest value. At initialization, we want diffuse weights so that every token contributes to learning. If softmax saturates immediately, only one token gets any gradient signal, and the rest are effectively dead.

The scaling formula

Multiplying the dot products by `1 / sqrt(head_size)` — equivalently, dividing by `sqrt(head_size)` — restores unit variance to the pre-softmax values regardless of head size. This keeps softmax in its diffuse regime at initialization, which is what we want. As training progresses and the model learns meaningful query/key alignments, the softmax can become peakier where appropriate — but it does so because the model decided to, not because of an initialization artifact.

Karpathy's framing

“It's really important especially at initialization that we be fairly diffuse. The problem is that because of softmax, if weight takes on very positive and very negative numbers inside it, softmax will actually converge towards one-hot vectors. Basically you're aggregating information from like a single node — every node just aggregates information from a single other node, that's not what we want especially at initialization. The scaling is used just to control the variance at initialization.”

PART IV

Building the Full Transformer

Multi-head attention, feedforward layers, residual connections, LayerNorm, and the pre-norm formulation that makes deep stacks train.

18. Multi-head attention in parallel

Chapter overview

A single attention head learns one type of token-to-token communication. But there are many types of useful communication — one head might learn syntactic relationships (verbs to subjects), another might learn semantic relationships (pronouns to antecedents), another might learn positional relationships (adjacent tokens). Multi-head attention runs several attention heads in parallel and concatenates their outputs, giving the model multiple independent communication channels.

Cell - The MultiHeadAttention module

```
class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """
    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(n_embd, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out
```

Reading the implementation

- `nn.ModuleList` — a container that holds the heads and registers them as proper PyTorch submodules so their parameters appear in `model.parameters()`.
- `torch.cat([h(x) for h in self.heads], dim=-1)` — run all heads on the same input, get back a list of (B, T, head_size) tensors, and concatenate along the channel dimension to produce (B, T, num_heads * head_size).
- `self.proj` — a final linear projection. This is necessary so the concatenated output of the heads can be mixed and shaped to fit back into the residual stream (Chapter 21).
- `self.dropout` — optional dropout for regularization, applied after the projection.

Choosing head_size

If we want num_heads heads and we want the concatenated output to be n_embd-dimensional (so it can fit back into the residual stream), then head_size = n_embd // num_heads. For our setup with n_embd = 32 and num_heads = 4, head_size = 8. For the final scaled-up model with n_embd = 384 and num_heads = 6, head_size = 64. The total “compute” is similar to one large attention head, but it is split across multiple smaller heads with independent parameters.

The group convolution analogy

“This is kind of similar to, if you're familiar with convolutions, this is kind of like a group convolution — because basically instead of having one large convolution, we do convolution in groups. And that's multi-headed self-attention.” The shared structure is: split the channels into groups, do something separately within each group, then re-combine. In multi-head attention, the “something” is self-attention with its own learned key/query/value projections.

19. The position-wise feedforward layer

Chapter overview

After multi-head attention has done the communication step, the model needs computation. The position-wise feedforward layer is a small MLP applied independently to each position. It does no token-to-token communication; it just lets each token “think” about what it received from the attention step. This is the second half of every Transformer block.

Cell - The FeedForward module

```
class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)
```

Why 4x expansion in the middle

The hidden layer of the feedforward is *four times* the embedding dimension. This is the convention from the original paper and has stuck in nearly every Transformer since. For our model with n_embd = 32, the inner layer is 128-dimensional. For the scaled-up model with n_embd = 384, the inner layer is 1536-dimensional. The expansion gives the feedforward layer significant computational capacity — in fact, the feedforward weights typically dominate the parameter count of a Transformer, more than the attention weights.

Position-wise means: applied independently

The same linear layers are applied to every position in parallel. Position 0 does not see position 1 during feedforward; they are completely independent. This is why attention does the communication step — without attention, the tokens would never interact at all. Attention is communication, feedforward is computation. They alternate in every block.

20. The Block: communication then computation

Chapter overview

A Transformer block bundles one multi-head attention layer (communication) with one feedforward layer (computation). The full Transformer is just a stack of these blocks, applied sequentially. This chapter introduces the Block class and the alternating communication-then-computation pattern that defines the architecture.

Cell - The Block class (initial version, before residual)

```
class Block(nn.Module):
    """ Transformer block: communication followed by computation """
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)

    def forward(self, x):
        x = self.sa(x)
        x = self.ffwd(x)
        return x
```

Each block first does self-attention (the tokens communicate), then does feedforward (each token does some independent computation). The forward pass is just two function calls, applied in sequence. The block can then be repeated as many times as we want, stacked in a `nn.Sequential`.

Cell - Stacking blocks

```
self.blocks = nn.Sequential(
    Block(n_embd, n_head=4),
    Block(n_embd, n_head=4),
    Block(n_embd, n_head=4),
)
```

> **The depth problem**
> Karpathy tries to train this version — communication, then computation, repeated three times — and the result is bad. “The problem is we're starting to actually get like a pretty deep neural net, and deep neural nets suffer from optimization issues, and I think that's what we're kind of like slightly starting to run into. So we need one more idea that we can borrow from the Transformer paper to resolve those difficulties.” That idea is residual connections.

21. Residual connections and the depth problem

Chapter overview

Deep neural networks suffer from a fundamental optimization problem: gradients have to flow backward through every layer, and each layer can shrink or distort them. By the time gradients reach the early

layers, they may be too small or too noisy to drive meaningful learning. Residual connections solve this by giving gradients a shortcut — an unimpeded “super-highway” from the loss back to the input. This is the single most important architectural trick for training deep networks, introduced by He et al. in the ResNet paper (2015).

How residual connections work

Cell - Block with residual connections

```
class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)

    def forward(self, x):
        x = x + self.sa(x)      # residual connection around attention
        x = x + self.ffwd(x)    # residual connection around feedforward
        return x
```

The change is tiny but transformative. Instead of `x = self.sa(x)`, we write `x = x + self.sa(x)`. The attention layer now computes a *residual* — how to modify the input — rather than replacing it. Same for the feedforward layer. The input flows through the network largely unchanged, with each block adding small corrections.

> **Why this helps gradients**
> Recall from Part 4 of the makemore series: addition distributes gradients equally to both branches. So the gradient from the loss flows back along two paths at each block — one through the residual blocks (attention, feedforward) and one straight through the addition. The straight-through path is the “gradient super-highway” that preserves the gradient signal regardless of what happens inside the block. At initialization, the blocks are typically configured so they contribute very little, meaning the network behaves almost like the identity function. Training then gradually brings the blocks online.

The projection inside MultiHeadAttention

The `self.proj` linear layer at the end of MultiHeadAttention exists specifically because of residual connections. Its job is to project the concatenated head outputs back into the residual stream so that the dimensions match for the `x + sa(x)` addition. Similarly, the feedforward layer ends with a projection from 4*n_embd back down to n_embd. Karpathy implements that projection as the second linear layer inside the feedforward, so it is implicit there.

The result of adding residuals

“I train this and we actually get down all the way to 2.08 validation loss, and we also see that the network is starting to get big enough that our train loss is getting ahead of validation loss — so we're starting to see a little bit of overfitting. And our generations here are still not amazing, but at least you see that we can see ‘is here’ — this now ‘grief syn’ like — this starts to almost look like English.”

22. LayerNorm: normalizing rows instead of columns

Chapter overview

Residual connections fix the gradient flow problem but introduce a new one: the magnitudes of activations can grow uncontrollably as they pass through many blocks (each block adds to the existing signal). Normalization is the standard fix. The original Transformer paper uses LayerNorm — a close cousin of the BatchNorm we built in Part 3 of makemore — but normalizing along a different axis.

BatchNorm vs LayerNorm

Recall that BatchNorm normalizes *columns* — for each feature dimension separately, it makes the mean and variance unit across the batch. LayerNorm does the opposite: it normalizes *rows* — for each individual example separately, it makes the mean and variance unit across the features.

Cell - LayerNorm by modifying BatchNorm

```
class LayerNorm1d:   # (used to be BatchNorm1d)
    def __init__(self, dim, eps=1e-5, momentum=0.1):
        self.eps = eps
        self.gamma = torch.ones(dim)
        self.beta = torch.zeros(dim)

    def __call__(self, x):
        xmean = x.mean(1, keepdim=True)        # changed from dim=0 to dim=1
        xvar = x.var(1, keepdim=True)           # changed from dim=0 to dim=1
        xhat = (x - xmean) / torch.sqrt(xvar + self.eps)
        self.out = self.gamma * xhat + self.beta
        return self.out

    def parameters(self):
        return [self.gamma, self.beta]
```

What changed from BatchNorm

- **The reduction dimension** — from 0 (batch) to 1 (features). This is the only line-level change.
- **No running statistics needed** — because normalization happens within each example, there is no train/eval distinction. The same code runs at training and inference.

- **No buffers, no momentum** — the bookkeeping that made BatchNorm complicated is all gone.
- **Still has learnable gamma and beta** — just like BatchNorm, after normalization we apply a learned scale and shift so the network can undo the normalization if it wants to.

> **Why LayerNorm works better for sequences**
> BatchNorm's per-feature statistics depend on the batch — the same example produces different normalized values depending on what other examples happen to be in the minibatch. For sequence models this is awkward: at inference time we may want to process a single sequence at a time, but BatchNorm needs a batch. LayerNorm sidesteps the issue entirely — each example normalizes itself. There is no train/eval distinction and no need for running statistics. This is why every modern Transformer uses LayerNorm (or its descendants like RMSNorm).

23. The pre-norm formulation and the final architecture

Chapter overview

The original Attention Is All You Need paper applies LayerNorm *after* each sublayer (attention and feedforward), in what is called the post-norm formulation. Over the years, the community has converged on a slightly different pattern: applying LayerNorm *before* each sublayer. This is called the pre-norm formulation and is what every modern LLM uses, including GPT-2, GPT-3, and Llama. This chapter explains the difference and shows the final Block implementation.

Cell - The Block with pre-norm LayerNorm

```
class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))      # pre-norm: LayerNorm before attention
        x = x + self.ffwd(self.ln2(x))    # pre-norm: LayerNorm before feedforward
        return x
```

Notice the structure: the residual stream `x` is added to the output of `sa(ln1(x))`, not `ln1(sa(x))`. LayerNorm normalizes the input *before* it goes into the sublayer; the sublayer operates on the normalized input; the output is added back to the un-normalized residual. This keeps the residual stream itself uncluttered — gradients flow through the additions without being touched by LayerNorm.

Why pre-norm trains more reliably

In post-norm (the original paper), the LayerNorm is on the residual stream itself, which means gradients have to flow back through it. In pre-norm, the residual stream passes through additions only, never through LayerNorm. This gives an even cleaner gradient highway, and in practice pre-norm Transformers train more stably at large depth. The original paper has been updated in many implementations — you will see both formulations in the wild, but pre-norm is now standard for new architectures.

Putting all the pieces together

Cell - The final Transformer language model

```
class BigramLanguageModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_embedding_table    = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[
            Block(n_embd, n_head=n_head) for _ in range(n_layer)
        ])
        self.ln_f = nn.LayerNorm(n_embd)              # final LayerNorm
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        tok_emb = self.token_embedding_table(idx)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device))
        x = tok_emb + pos_emb
        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)
        ...
```

This is the complete decoder-only Transformer. Token embeddings plus positional embeddings flow into a stack of `n_layer` Transformer blocks. After the blocks, a final LayerNorm normalizes the activations before the language modelling head projects to vocab-sized logits. The whole architecture is about 50 lines of code excluding the helper classes, and structurally identical to GPT-2 and GPT-3.

A final note on the final LayerNorm

“One more thing I forgot to add is that there should be a layer norm here also, typically as at the end of the Transformer and right before the final linear layer that decodes into vocabulary.” The final `ln_f` normalizes the output of the last block before projecting to logits. This is a small detail but appears in essentially every modern Transformer implementation.

PART V

Scaling, ChatGPT Context, and Reference

Hyperparameter scaling to 10M parameters, dropout regularization, the encoder/decoder distinction, the full ChatGPT pipeline, and reference materials.

24. Scaling up: hyperparameters that matter

Chapter overview

The architecture is now complete. To get good results we need to scale it up. Karpathy increases the embedding dimension, the number of layers, the number of heads per layer, the batch size, and the context length. This chapter walks through the final hyperparameter settings and the validation loss those settings achieve.

Cell - Final hyperparameters

```
# hyperparameters
batch_size = 64
block_size = 256       # context length (was 8 earlier)
max_iters = 5000
eval_interval = 500
learning_rate = 3e-4   # lowered because the model is bigger
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 384           # was 32 earlier
n_head = 6
n_layer = 6
dropout = 0.2
```

What each hyperparameter controls

- **batch_size = 64** — how many independent contexts we process per training step. Larger is better for gradient quality, limited by GPU memory.
- **block_size = 256** — the maximum context length. The model can attend to up to 256 previous characters when predicting the next one. Up from just 8 in the earlier examples.
- **n_embd = 384** — the dimension of the token embeddings and the residual stream. The width of the entire network.
- **n_head = 6, n_layer = 6** — six attention heads per block, six blocks stacked. With n_embd = 384, each head has size 384/6 = 64. This is the standard head_size in the Transformer literature.
- **learning_rate = 3e-4** — the “Karpathy constant.” A good default for Transformers of moderate size. Lower than the 1e-3 we used for the bigram because the bigger model is more sensitive.
- **dropout = 0.2** — randomly drops 20% of activations during training. Regularization against overfitting (Chapter 25).

The result

Configuration	Train loss	Val loss
Bigram baseline	~2.5	~2.5

+ single attention head	~	2.40
+ multi-head attention (4 heads)	~	2.28
+ feedforward layers	~	2.24
+ residual connections + 4x feedforward	~	2.08
+ LayerNorm (pre-norm formulation)	~	2.06
+ scale up (10M params, dropout 0.2)	**~**	**1.48**

Final validation loss **1.48**, down from 2.5 in the bigram baseline. The model now has about 10 million parameters and trains for 5000 iterations — roughly 15 minutes on an A100 GPU. The generated text is recognizably Shakespearean in structure (line breaks, dialogue tags, characters), even though the words and phrases are still nonsensical at the character level.

> **Karpathy's framing of the scale-up**
> "Drum roll — how well does it perform? Let me just scroll up here. We get a validation loss of 1.48, which is actually quite a bit of an improvement on what we had before, which I think was 2.07. So it went from 2.07 all the way down to 1.48 just by scaling up this neural net with the code that we have. And this of course ran for a lot longer — this maybe trained for I want to say about 15 minutes on my A100 GPU."

Two infrastructure helpers introduced for scaling

When the model gets this large, two more pieces of training-loop infrastructure become important. Both are small but necessary additions to the bigram-era training code from Chapter 6.

Helper 1: the estimate_loss function

Single-step loss values are noisy — we saw this in Part 5 of the makemore series. A loss of 1.5 on one batch and 1.6 on the next does not necessarily mean training regressed; it might just be batch-to-batch noise. For larger models we want a more stable estimate of the loss, computed by averaging over multiple batches. `estimate_loss` does exactly that.

Cell - The estimate_loss helper

```
@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
```

```
        out[split] = losses.mean()
    model.train()
    return out
```

- `@torch.no_grad()` — the decorator disables autograd graph construction inside the function. We are not going to backprop through this loss, so we save memory and compute.
- `model.eval()` — switches Dropout off and LayerNorm into eval mode. Important because BatchNorm and Dropout behave differently at training and inference. (LayerNorm does not actually have a different mode, but the call is still standard.)
- **Average over** `eval_iters` **batches** — computes `eval_iters` separate loss values and averages them, smoothing out batch-to-batch noise.
- `model.train()` at the end — switches back to training mode before returning, so the next training step works correctly.

The training loop calls `estimate_loss()` every `eval_interval` iterations to get a stable read on how training is progressing on both train and val splits. This is how Karpathy gets the clean loss curves shown in the lecture.

Helper 2: device handling

Training a 10-million-parameter Transformer on a CPU is impractically slow. On a GPU the same training runs in about 15 minutes. To move computation to the GPU, the model parameters and the input tensors all need to be moved to the GPU device. PyTorch makes this clean with the `.to(device)` pattern.

Cell - Device handling

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'

# In get_batch:
x, y = x.to(device), y.to(device)

# After creating the model:
model = BigramLanguageModel()
m = model.to(device)

# When using the position embeddings:
pos_emb = self.position_embedding_table(torch.arange(T, device=device))
```

> **What .to(device) does**
> Calling `.to(device)` on a tensor or module moves it to the specified device (CPU or GPU). For modules, this recursively moves every parameter and registered buffer. Tensors that interact during the forward pass must all live on the same device, so we have to move both the model and the input data. Forgetting to move any one piece produces the famous “expected all tensors to be on the same device” error — the canonical first GPU bug.

25. Dropout as regularization

Chapter overview

Dropout is the standard regularization technique for large neural networks. Introduced by Srivastava et al. in 2014, it works by randomly zeroing out a fraction of activations during every forward pass. This forces the network to not rely too heavily on any single neuron, effectively training an ensemble of sub-networks. We add it in three places in the Transformer.

How dropout works

During training, dropout randomly sets a fraction of its inputs to zero. The rest are scaled up to compensate (multiplied by `1 / (1 - p)`) so the expected value of the output stays the same. At inference time, dropout is disabled and all activations flow through normally. The dropout mask changes on every forward pass, so the network never sees the same masked sub-network twice.

Where we apply dropout

- **Inside each attention head**, applied to the attention weights after softmax. Randomly prevents some token-to-token communications.
- **At the output of multi-head attention**, applied after the projection back to n_embd. Regularizes the contribution of the attention block to the residual stream.
- **At the output of the feedforward layer**, applied after the final linear. Regularizes the contribution of the feedforward block to the residual stream.

> **The ensemble interpretation**
> "Because the mask of what's being dropped out is changed every single forward-backward pass, it ends up kind of training an ensemble of sub-networks. And then at test time, everything is fully enabled and kind of all of those sub-networks are merged into a single ensemble." This is the standard intuition for why dropout helps generalization. The actual mathematical analysis is more subtle, but the ensemble framing captures the essential idea.

Dropout becomes important when scaling up the model. With n_embd = 384, n_layer = 6, and 6 heads per block, the network has enough capacity to overfit the 1-million-character training set if left unregularized. The 0.2 dropout rate prevents that — the train loss and val loss stay close together throughout training.

26. Encoder vs decoder, cross-attention, and nanoGPT

Chapter overview

The original Attention Is All You Need paper presents an encoder-decoder Transformer, designed for machine translation. The Transformer we have built is a *decoder-only* variant, which is what GPT uses. This chapter explains the distinction, introduces cross-attention (the missing piece in our model that the encoder-decoder paper has), and points to nanoGPT — Karpathy's actual GPT-2 reproduction codebase

— for further reference.

The decoder side: what we built

Our model has multi-head self-attention with triangular masking, feedforward layers, residual connections, and LayerNorm. The triangular masking is what makes it a *decoder* — tokens can only see past tokens, which is required for autoregressive generation. At inference time we feed in some context, predict the next token, append it to the context, and repeat. The model has nothing to condition on besides the prefix itself.

The encoder side: what we did not build

An *encoder* block is the same as a decoder block but without the triangular mask. All tokens can attend to all other tokens, in both directions. Encoders are used when the input is fixed and known in advance, like in BERT (text classification) or in the encoder of a translation model (the source language sentence).

Cross-attention: how encoders connect to decoders

In the original encoder-decoder Transformer, the decoder has one extra attention layer per block that is not self-attention: *cross-attention*. In cross-attention, the queries come from the decoder's residual stream, but the keys and values come from the *encoder's* final output. This lets the decoder, while generating an English translation, look at the encoded French sentence.

> **Self-attention vs cross-attention**
>
> **Self-attention.** Keys, queries, and values all come from the same source. Used in both encoders and decoders.
>
> **Cross-attention.** Queries come from one source (the decoder), keys and values come from another (the encoder's output). Used only in encoder-decoder models. Allows conditioning the decoder on encoded context.

A concrete example: French-to-English translation

To make the encoder-decoder distinction concrete, consider what the original Attention Is All You Need paper was actually built for: machine translation. The setup is asymmetric — we have a French sentence as input and need to produce an English translation. The encoder reads the French, the decoder writes the English.

Concrete encoder-decoder setup for translation

```
# <----------- ENCODE -----------><------------- DECODE ---------------->
# les reseaux de neurones sont geniaux! <START> neural networks are awesome!<END>
#
# Encoder: processes the French sentence bidirectionally.
#          No mask - every French token attends to every other French token.
#          Produces a sequence of encoded vectors that summarize the source.
#
# Decoder: generates the English translation one token at a time.
```

```
#           Has triangular masking on its self-attention (autoregressive).
#           Each layer ALSO does cross-attention to the encoder's output,
#           letting it look at the French while writing the English.
```

- **The encoder side processes the French input.** All French tokens can attend to each other in both directions, with no masking. The encoder produces a sequence of encoded vectors that the decoder will later use.
- **The decoder generates English starting with a special <START> token** and continues autoregressively until it produces <END>. The decoder's self-attention is triangular-masked so each generated token only sees previously-generated tokens.
- **The decoder's cross-attention layer is the bridge between the two.** Queries from the decoder's residual stream are matched against keys from the encoder's output, and values are aggregated from the same encoder output. This is how the decoder "reads" the French while writing the English.

> **Why we built decoder-only**
> "We did not do this because we have nothing to encode — there's no conditioning, we just have a text file and we just want to imitate it. And that's why we are using a decoder-only Transformer, exactly as done in GPT." Modern LLMs (GPT-3, ChatGPT, Claude, Llama) are all decoder-only. The encoder-decoder split has largely fallen out of favor because pre-training a single decoder on a massive corpus turns out to be more flexible than maintaining two separate components.

The nanoGPT codebase

Karpathy has a public GitHub repository called nanoGPT that reproduces the GPT-2 architecture in about 300 lines of model code and 300 lines of training code. The model file is structurally identical to what we built in this lecture, with two main implementation differences:

- **Batched multi-head attention.** Instead of running each head as a separate Head instance and concatenating, nanoGPT reshapes the input to add a head dimension and runs all heads with a single batched matmul. Same math, much more efficient in practice.
- **GELU instead of ReLU.** The feedforward uses GELU as the activation function, matching what OpenAI used in GPT-2. Slightly smoother and tends to give marginally better results.

If you want to scale beyond the toy model in this lecture — load pretrained GPT-2 weights, train on larger datasets, run on multi-GPU clusters — nanoGPT is the natural next step.

27. From pretraining to ChatGPT: SFT and RLHF

Chapter overview

Training a base Transformer (what we did in this lecture) gets you a document completer — a model that imitates whatever it was trained on. It does not get you an assistant. To go from a base model to ChatGPT, OpenAI runs the base model through two more training stages: *supervised fine-tuning* (SFT)

and *reinforcement learning from human feedback* (RLHF). This chapter explains both stages at a high level.

Stage 1: Pretraining (what we built)

Train a decoder-only Transformer on a large unlabeled corpus to predict the next token. Our model: 10M parameters, 1M characters of Shakespeare. GPT-3: 175B parameters, 300B tokens from a curated mix of internet text. The architectures are nearly identical — the difference is scale (about 17,500x more parameters and 1,000,000x more training data) and infrastructure (thousands of GPUs running in parallel, training for weeks or months). The resulting base model can complete documents but does not specifically answer questions or follow instructions.

Side-by-side: our model vs GPT-3

Property	This lecture	GPT-3 (largest)
Parameters	~10 million	175 billion
Training tokens	~1 million chars (~300K BPE tokens)	300 billion tokens
Vocabulary	65 (chars)	~50K (BPE)
Layers (n_layer)	6	96
Embedding dim (n_embd)	384	12,288
Attention heads (n_head)	6	96
Head size	64	128
Context window	256 chars	2048 tokens
Batch size	64	3.2M tokens
Training time	~15 min on 1 GPU	Weeks on thousands of GPUs

The architecture is the same; the scale is what's different

“The architecture is actually like nearly identical to what we implemented ourselves. But of course it's a massive infrastructure challenge to train this. You're talking about typically thousands of GPUs having to talk to each other to train models of this size. So that's just the pretraining stage.” Every row in the table above is just a hyperparameter knob; the model class definition would barely change. What changes is the engineering.

Stage 2: Supervised fine-tuning (SFT)

Collect a smaller dataset (thousands of examples) of high-quality question-answer pairs written by human labelers. Fine-tune the base model on this dataset using the same next-token-prediction objective. The base model already "knows" how to write coherent text from pretraining; SFT just nudges it to produce text in the question-answer format. After SFT, the model produces assistant-like responses by default, but the quality is variable.

Stage 3: Reinforcement learning from human feedback (RLHF)

- **3a. Reward model training.** Sample multiple responses from the SFT model for the same prompt. Have human labelers rank them by preference. Train a separate reward model to predict the human ranking.
- **3b. Policy optimization.** Use the reward model to score samples from the language model, then use Proximal Policy Optimization (PPO) or a similar policy-gradient method to update the language model so it produces higher-reward samples.

The result is a model whose samples align more closely with what humans actually want. ChatGPT was the public-facing product that demonstrated this pipeline works on a large scale. The alignment data is generally not publicly available, so this stage is harder to replicate than pretraining (where the data is public and the model code is well-understood).

> **What this lecture covers vs what it doesn't**
> "NanoGPT focuses on the pretraining stage. ... If you're interested in something that's not just language modelling but you actually want to perform tasks, or you want them to be aligned in a specific way, or you want to detect sentiment or anything like that — basically anytime you don't want something that's just a document completer — you have to complete further stages of fine-tuning, which we did not cover."

28. Diagnostic cards: Transformer pitfalls

Chapter overview

Building a Transformer from scratch surfaces a specific set of recurring bugs and footguns. This chapter consolidates them into stand-alone diagnostic cards. Keep these handy when implementing attention-based models from scratch — almost every Transformer bug you will encounter falls into one of these categories.

Pitfall 1 · Cross-entropy shape mismatch	
Symptom	`F.cross_entropy` throws a shape error when called on (B, T, C) logits and (B, T) targets, or runs but produces incorrect loss values.
Diagnosis	PyTorch's cross-entropy expects channels as the *second* dimension for multi-dimensional inputs — (B, C, T) not (B, T, C). Our convention is the latter, so we have a mismatch.
Action	Reshape before passing to cross-entropy: `logits.view(B*T, C)` and `targets.view(B*T)`. This flattens the batch and time dimensions into a single dimension and avoids the convention mismatch entirely.

Pitfall 2 · Future-leakage in attention	
Symptom	Training loss drops to near zero very quickly. Generated text at inference time is gibberish despite excellent training metrics.
Diagnosis	The triangular mask is missing or applied incorrectly. The model has learned to copy from future positions during training, which gives perfect training loss but is meaningless because future positions are unavailable at inference.
Action	Always apply `wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))` before softmax in every attention layer. Verify by printing one row of `wei` after softmax — row i should have zeros in positions i+1 through T-1.

Pitfall 3 · Softmax saturation at initialization	
Symptom	Training loss does not decrease meaningfully. Attention weights look near one-hot (all weight on one token) from the very first step.
Diagnosis	Missing the `* head_size ** -0.5` scaling factor. The dot products between random keys and queries have variance proportional to head_size, which makes softmax saturate immediately.
Action	Always scale the affinities by `head_size ** -0.5` before masking and softmax. This keeps the pre-softmax values at unit variance regardless of head size.

Pitfall 4 · Position embedding out of bounds	
Symptom	Runtime error during generation: `IndexError: index out of range in self` from the position embedding table.
Diagnosis	The position embedding table has only `block_size` entries. During generation, after generating block_size+1 tokens, the position index exceeds the embedding table size.
Action	Crop the context to the last block_size tokens before each forward pass during generation: `idx_cond = idx[:, -block_size:]`. The model is autoregressive, so the older tokens beyond the context window are simply forgotten.

Pitfall 5 · Vanishing gradients in deep stacks	
Symptom	Loss plateaus at a poor value when adding more Transformer blocks. Removing blocks (going back to 2-3) actually improves performance.
Diagnosis	Missing residual connections. Without them, gradients shrink by a factor of less-than-one at each block, becoming useless by the time they reach early layers.
Action	Wrap every sublayer (attention, feedforward) in a residual connection: `x = x + sublayer(ln(x))`. This gives gradients a direct path from loss to input that bypasses the sublayer transformations.

Pitfall 6 · Tril buffer not on device	
Symptom	Runtime error: `Expected all tensors to be on the same device` when moving model to GPU.
Diagnosis	The `tril` matrix was created as a regular tensor attribute, not registered as a buffer. PyTorch's `.to(device)` only moves parameters and registered buffers, not arbitrary attributes.
Action	Use `self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))` in the Head's __init__. Buffers move with the model to whatever device the model is on.

29. Summary and where to go next

Chapter overview

Time to consolidate. This chapter lists the most important takeaways from the lecture and suggests directions for further study.

The ten things to take away

- **1. Attention is communication, feedforward is computation.** The Transformer alternates between letting tokens talk to each other (attention) and letting each token process what it received (feedforward).
- **2. The mathematical trick of attention is weighted aggregation via matrix multiplication.** A lower-triangular matrix multiplied with the input produces a weighted average of past positions. Softmax with -inf masking is the universal way to express this.
- **3. Keys, queries, and values are three separate learned projections.** The query is what a token is looking for; the key is what it offers; the value is what it actually communicates. These three roles are conceptually distinct.
- **4. Scaled dot-product attention requires the /sqrt(d_k) scaling.** Without it, softmax saturates into one-hot at initialization, killing learning.
- **5. Multi-head attention runs several attention layers in parallel.** Different heads learn different types of token-to-token communication, then concatenated.
- **6. Residual connections are how you train deep networks.** Gradient super-highway from the loss back to the input, with residual blocks gradually coming online.
- **7. LayerNorm normalizes rows, not columns.** Each example normalizes itself across its features. No batch dependence, no train/eval distinction, no running statistics.
- **8. Pre-norm is the modern formulation.** LayerNorm goes *before* the sublayer, not after. The residual stream itself stays unnormalized.
- **9. Decoder-only architectures use triangular masking; encoders do not.** The choice depends on whether tokens can see the future. Language modelling uses decoders.
- **10. Pretraining gives you a document completer; SFT and RLHF turn it into an assistant.** The architecture is the easy part; the alignment is what makes ChatGPT useful.

Where to go next

- **nanoGPT.** Karpathy's GPT-2 reproduction. Same architecture as this lecture, but production-grade with multi-GPU training and pretrained weight loading. Start here.
- **Read the original papers.** “Attention Is All You Need” (2017), “Improving Language Understanding by Generative Pre-Training” (GPT, 2018), “Language Models are Few-Shot Learners” (GPT-3, 2020).
- **Build a tokenizer.** The character-level tokenization in this lecture is the simplest possible. Real LLMs use BPE or SentencePiece. Karpathy has a separate “tokenizer” lecture covering this.

- **Scale up.** Train nanoGPT on OpenWebText (a public dataset comparable to what GPT-2 was trained on). With a single A100 GPU and a few days, you can reproduce GPT-2 124M.
- **Explore alignment.** Reinforcement Learning from Human Feedback is the rabbit hole that leads to ChatGPT. PPO, reward modeling, Constitutional AI, DPO — the field is moving fast.
- **Try different architectures.** RoPE (rotary position embeddings), grouped-query attention, mixture of experts, sliding-window attention. The Transformer is not the final word.

> **Karpathy's send-off**
> "That kind of like concludes the programming section of this video. We basically kind of did a pretty good job of implementing this Transformer. ... I hope you enjoyed the lecture and uh yeah, go forth and transform."

30. Glossary

This glossary defines every technical term used in the handbook in a single, scannable place. Look up anything you do not recognize here first.

Affinity. The pre-softmax interaction strength between two tokens in attention. Computed as the dot product of one token's query with another's key. Large positive affinity means the first token finds the second highly relevant; large negative affinity means it actively wants to ignore the second.

AdamW. A variant of the Adam optimizer with decoupled weight decay. The default optimizer for Transformer training. Maintains per-parameter running estimates of gradient mean and variance to scale each parameter's update individually. Much more efficient than plain SGD on Transformer loss surfaces.

Attention. A communication mechanism between tokens in which each token aggregates information from other tokens via a weighted sum, with the weights determined by data-dependent affinities. Implemented as scaled dot-product attention in Transformers.

Autoregressive. A model that generates output one token at a time, conditioning each prediction on all previous predictions. GPT and our decoder-only Transformer are autoregressive.

Batch dimension (B). The first dimension of input tensors. Holds independent training examples that share parameters but are processed in parallel. Each batch element is independent — tokens in one batch element never see tokens in another.

Bigram model. A language model that predicts the next token based only on the current token (no longer-range context). Our starting baseline before adding attention. About 2.5 loss on tiny Shakespeare.

Block (Transformer block). The unit that the Transformer stacks repeatedly: a multi-head attention layer followed by a feedforward layer, both wrapped in residual connections and preceded by LayerNorm (in the pre-norm formulation).

Block size (block_size). The maximum context length the model can attend to. The number of position embeddings. In our final model, 256. In GPT-3, 2048. In GPT-4 and beyond, often 8K to 128K or more.

Causal mask. A lower-triangular mask applied to attention weights to prevent tokens from attending to future positions. Required for autoregressive generation. Equivalent terms: triangular mask, autoregressive mask.

Channel dimension (C). The last dimension of input tensors. Holds the features (the components of each token's representation). Our embedding dimension n_embd is the size of this dimension.

Character-level tokenization. Encoding text by mapping each character to an integer. The simplest tokenization scheme. Vocabulary size is small (65 for tiny Shakespeare) and sequences are long.

ChatGPT. The conversational AI product released by OpenAI in late 2022. Built on a base GPT model (pretraining), then refined through supervised fine-tuning and reinforcement learning from human feedback. The product launch that made modern LLMs mainstream.

Cross-attention. Attention in which the queries come from one source (the decoder) but the keys and values come from a separate source (the encoder's output). Used in encoder-decoder Transformers for translation. Not used in our decoder-only model.

Decoder. A Transformer (or block) with triangular masking, used for autoregressive generation. Our model is decoder-only. GPT-style.

Dropout. A regularization technique that randomly zeros out a fraction of activations during training. Forces the network to not depend too heavily on any single neuron. Disabled at inference time.

device (.to(device)). PyTorch's mechanism for specifying which device (CPU, single GPU, or specific GPU index) tensors and modules live on. Tensors that interact during the forward pass must all be on the same device. `model.to('cuda')` recursively moves every parameter and registered buffer to the GPU.

Embedding. A learned mapping from discrete tokens (or positions) to dense vector representations. `nn.Embedding` is just a lookup table whose rows are the embeddings.

Encoder. A Transformer (or block) without triangular masking, in which all tokens can attend to all other tokens bidirectionally. Used for classification and as the input side of encoder-decoder models.

estimate_loss. A utility function in the final training script that averages the loss over `eval_iters` batches to produce a stable estimate. Wrapped in `@torch.no_grad()` and toggles the model between `eval()` and `train()` modes around the evaluation. Used to print clean train/val loss values every `eval_interval` training steps.

Feedforward layer. A small MLP applied independently to each position. The computation step in a Transformer block. Typically expands to 4x the embedding dimension in the middle and contracts back.

GELU. Gaussian Error Linear Unit. A smoother variant of ReLU used in GPT-2 and many modern Transformers. We use ReLU in this lecture for simplicity.

Head (attention head). A single instance of self-attention, with its own learned key/query/value projections and a head_size-dimensional output. Multi-head attention concatenates several heads.

Head size. The dimensionality of each individual attention head's output. Typically n_embd / n_head so the concatenated multi-head output is n_embd-dimensional.

Key (K). One of the three learned projections in attention. Conceptually, “what this token offers.” Dotted with other tokens' queries to compute affinities.

LayerNorm. Normalization along the feature axis. Each example normalizes itself across its features (rows), unlike BatchNorm which normalizes across the batch (columns). Stateless, no train/eval distinction.

LM head (language modeling head). The final linear layer that projects from the residual stream (n_embd-dimensional) up to vocab-sized logits. Typically the only place vocab_size appears in the model architecture.

Multi-head attention. Several attention heads running in parallel on the same input, with their outputs concatenated channel-wise and projected back to n_embd. Different heads can learn different types of token-to-token communication.

nanoGPT. Karpathy's GitHub repository that reproduces GPT-2 in about 600 lines of code. Production-grade version of the model built in this lecture.

Positional embedding. A learned vector for each position in the sequence, added to the token embedding. Tells the model where in the sequence each token is. Necessary because attention itself has no notion of position.

Pre-norm formulation. The arrangement of layers where LayerNorm is applied to the input *before* the sublayer, and the output of the sublayer is added to the residual stream. `x = x + sublayer(ln(x))`. Modern standard.

Pretraining. The first stage of training a large language model: predict the next token on a large unlabeled corpus. Produces a base model that can complete documents but is not yet aligned to follow instructions.

Query (Q). One of the three learned projections in attention. Conceptually, “what this token is looking for.” Dotted with other tokens' keys to compute affinities.

Register buffer. A PyTorch mechanism for attaching non-parameter tensors (like the tril mask) to a module. Buffers move with the module to whatever device the module is on, but are not updated by the optimizer.

Residual connection. An addition of a layer's input to its output, so the layer learns a residual transformation. Gives gradients a direct path from loss to input. The single most important trick for training deep networks.

Residual stream. The information stream that flows through the Transformer, modified incrementally by each block via residual connections. Starts as embeddings, ends as the input to the LM head.

RLHF. Reinforcement Learning from Human Feedback. The post-pretraining stage that turns a base model into an assistant. Three steps: collect preference data, train a reward model, optimize the language model against the reward model using PPO.

Scaled dot-product attention. The full attention formula: softmax(Q @ K^T / sqrt(d_k)) @ V. The sqrt(d_k) scaling is what makes it “scaled” and what prevents softmax saturation at initialization.

Self-attention. Attention in which the keys, queries, and values all come from the same source. As opposed to cross-attention, where queries come from one source and keys/values come from another.

SFT. Supervised Fine-Tuning. The second stage of training (after pretraining, before RLHF). Fine-tune the base model on a smaller dataset of high-quality question-answer pairs to nudge it toward assistant-like behavior.

Softmax. Converts a vector of real numbers into a probability distribution. Used in attention to convert affinities into normalized attention weights. Sensitive to input magnitudes — large inputs cause saturation.

Time dimension (T). The sequence dimension. Holds positions in the input sequence. Our final model has T = 256 (block_size). Each position is a token.

Tiny Shakespeare. Karpathy's favorite toy dataset: a 1-megabyte concatenation of Shakespeare's works. About 1.1 million characters, vocabulary of 65.

Tokenization. Converting raw text into a sequence of integer tokens. Character-level, word-level, or subword-level (BPE, SentencePiece, WordPiece). Our model uses character-level for simplicity.

Transformer. The neural network architecture introduced by Vaswani et al. (2017) in “Attention Is All You Need.” Defined by its use of multi-head attention. The basis of GPT, BERT, T5, and essentially every modern large language model.

Value (V). One of the three learned projections in attention. Conceptually, “what this token will communicate.” Aggregated according to the attention weights to produce the output.

Vocab size. The number of distinct tokens in the vocabulary. 65 in our character-level Shakespeare model. About 50,000 in OpenAI's tiktoken vocabulary. The size of the output layer of the LM head.

End of handbook. This is the lecture in Karpathy's Zero-To-Hero series that finally delivers the full Transformer — the architecture behind GPT-2, GPT-3, ChatGPT, Claude, Llama, and every other modern large language model. The makemore series built the fundamentals; this lecture put them together into something that can plausibly be scaled to ChatGPT. The next steps — tokenizers, scaling, alignment — are their own deep rabbit holes, and Karpathy has separate lectures and codebases for each. Go forth and transform.